Projet C++ : Simulateur de Bras Robotique Articulé

Pierre-Alexandre Monceau

Séance 1 – Conception UML et hiérarchie de joints

1. Classe abstraite CJoint :

   a. Pourquoi le destructeur doit-il être virtual ? Montrer le comportement indéfini sur un exemple concret (delete via pointeur de base)

Sans destructeur virtuel, supprimer un objet dérivé via un pointeur de base invoque uniquement le destructeur de la classe de base, laissant les ressources de la classe dérivée non libérées.

   b. Pourquoi setQ() ne peut-il pas être const ? Pourquoi les accesseurs getQ() etc. doivent-ils l'être ?

setQ() non const : un setter qui écrit dans le membre q_. Or, const sur une méthode garantit qu'aucun membre de l'objet n'est modifié.
Getter const : ils ne font que lire l'état de l'objet. Les marquer const permet de les appeler sur des objets ou références constants.

2. Classes dérivées :

   a. Ligature dynamique : Que se passe-t-il si getTransform() n'est pas virtuel ?

Sans virtual, la résolution est statique c'est-à-dire toujours la même, décidé à la compilation selon le type déclaré du pointeur, et non dynamique.

   b. Pourquoi ne peut-on pas utiliser le constructeur de copies par défaut directement depuis Cjoint ?

On ne peut pas utiliser directement le constructeur de copie par défaut de CJoint pour plusieurs raisons car CJoint est une classe abstraite non instanciable : le constructeur de copie par défaut tenterait de créer une instance de CJoint, ce qui est impossible puisqu'il s'agit d'une classe abstraite. De plus, une copie par défaut risquerait de tronquer l'objet en ne conservant que la partie correspondant à CJoint, en perdant les informations spécifiques aux classes dérivées. Enfin, les membres utilisant des pointeurs intelligents ne peuvent pas être copiés par défaut, ce qui rend le constructeur de copie implicite inadapté.

Séance 2 – CBras, STL et cinématique directe

1. Classe CBras :

a. Pourquoi addJoint() prend un unique_ptr par valeur et requiert std::move() à l'appel ?

unique_ptr est non copiable par conception. Passer par valeur force le transfert de propriété. Sans std::move(), la tentative de copie d'un unique_ptr ne compile pas.

b. Pourquoi CBras n'est-elle pas copiable par défaut ?

Parce que std::unique_ptr supprime son constructeur de copie. Par règle de propagation du compilateur, CBras hérite de cette suppression : son constructeur de copie généré par défaut essaierait de copier chaque unique_ptr du vecteur, ce qui est interdit.

c. Quel est le type de retour de operator<< et pourquoi retourner std::ostream&

retour : std::ostream&
Retourner std::ostream& permet le chaînage d'opérateurs.
std::ostream& plutôt que std::ofstream& pour une interface plus générique.

2. Cinématique directe :

a. Comparer A*B (Eigen) à une boucle triple manuelle. Citer deux avantages concrets

Performance : Eigen génère du code vectorisé et optimise l'accès mémoire. Pour des matrices 4×4 en boucle de cinématique, le gain est mesurable surtout si la boucle est appelée de nombreuses fois.

Exactitude numérique et maintenabilité : Eigen utilise des algorithmes numériquement stables et la syntaxe A*B est sans ambiguïté, sans risque d'erreur d'indice.

b. Pinocchio expose pinocchio::forwardKinematics(model,data,q). Quelle méthode de pinocchio::SE3 retourne la Eigen::Matrix4d équivalente ? Comment comparer les deux résultats numériquement ?

.toHomogeneousMatrix().
On utilise la norme de Frobenius sur la différence plutôt qu'une comparaison bit à bit, car les arrondis flottants entre deux calculs différents peuvent produire des écarts même pour des résultats mathématiquement identiques.
La méthode isApprox de la bibliothèque Eigen utilise cette norme.

Séance 3 – Règle des cinq, Eigen::VectorXd et template

1. Règle des cinq sur CBras.

   a. Constructeur de copie : Pourquoi ne pas copier le vecteur directement ?

Copier le vecteur directement est impossible : std::unique_ptr est non copiable par conception. Une copie directe provoquerait une erreur de compilation.

Il faut donc itérer et appeler clone() sur chaque élément, ce qui alloue dynamiquement un nouvel objet indépendant pour chaque joint. Cela garantit une copie profonde : les deux CBras possèdent leurs propres joints sans partage de pointeurs.

   b. Opérateur d'affectation par copie : idiome copy-and-swap, quelle garantie d'exception cela offre-t-il ?

Garantie : si la copie (construction temporaire) échoue, *this est inchangé. La phase swap n'admet pas d'exception, donc aucun état intermédiaire invalide n'est jamais observable.

   c. Constructeur et opérateur de déplacement : = default suffit-il ? Justifier en termes de unique_ptr.

Cela suffit car unique_ptr est déplaçable : son constructeur de déplacement transfère la propriété du pointeur brut vers le nouvel objet et met le pointeur source à nulle. Par propagation, le vecteur est lui aussi déplaçable. Le déplacement généré par défaut déplace donc le vecteur entier sans aucune allocation ni clonage.

   d. Destructeur : = default suffit-il ? Dans quel ordre les joints sont-ils détruits ?

Destructeur : = default suffit car il déclenche la destruction du membre joints_ (un vecteur) qui détruit chacun de ses éléments du plus grand indice au plus petit. Chaque élément étant des pointeurs intelligents qui libèrent automatiquement CJoint via delete. Aucune gestion manuelle de mémoire n'est requise.

2. Interface Eigen::VectorXd

a. Pourquoi Pinocchio utilise Eigen::VectorXd plutôt que std::vector<double> ? Mentionner deux opérations qui en bénéficient directement (norm(), dot(), . . .)

std::vector<double> est un conteneur générique conçu pour stocker des données, on ne peut pas lui appliquer des opérations directement. Eigen::VectorXd est un vecteur mathématique qui bénéficie des opérations vectorielles natives, de la compatibilité directe avec les algorithmes Pinocchio ou encore de l'utilisation d'instructions SIMD plus rapides. Opérations qui en bénéficient : q.norm() la norme euclidienne et q1.dot(q2) le produit scalaire.

3. Template CVecteur<T,N>

a. Pourquoi la définition entière doit-elle être dans le .h ?

Pour instancier un template, le compilateur a besoin du code complet au moment de la compilation. Il ne pourrait pas compiler les définitions des méthodes sans savoir leur type précis.

b. Différence entre le paramètre typename T et le paramètre non-type std::size_t N.

typename T est un alias de type : il disparaît à la compilation, remplacé par le type concret.

std::size_t N est une constante entière intégrée directement dans le code généré. Elle permet de fixer la taille du vecteur au moment de la compilation, sans allocation dynamique. C'est ce qui garantit que CVecteur<double, 3> et CVecteur<double, 4> sont deux types distincts et incompatibles.

4. Valgrind
valgrind --leak-check=full --track-origins=yes ./main

```
HEAP SUMMARY:
    in use at exit: 0 bytes in 0 blocks
  total heap usage: 12 allocs, 12 frees, 75,016 bytes allocated

All heap blocks were freed -- no leaks are possible

For lists of detected and suppressed errors, rerun with: -s
ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

D'après le rapport de Valgrind, il y n'y a pas de fuite mémoire. Le code est donc robuste.

Séance 4 – Intégration, validation Pinocchio

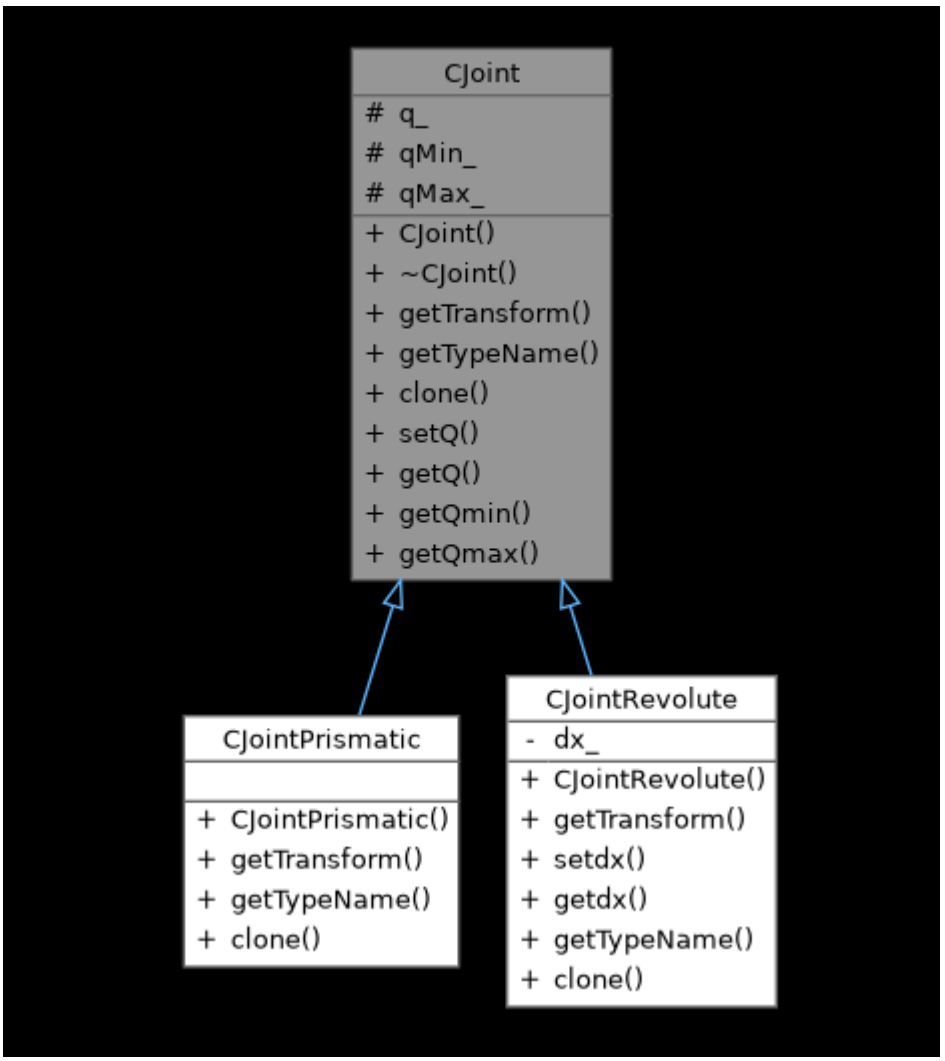
b. Pinocchio découple Model (lecture seule) et Data (mutable, résultats des algorithmes). Quel intérêt dans un contexte temps réel où plusieurs threads calculent en parallèle ?

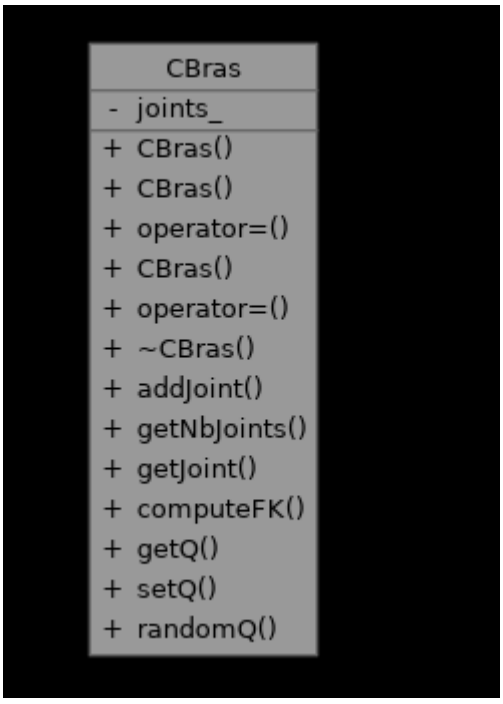
pinocchio::Model est en lecture seule (topologie, masses, limites) et peut être partagé entre threads sans mutex. pinocchio::Data contient tous les résultats mutables (oMi, v, a, ...). Chaque thread possède son propre Data : zéro contention, pas de copie du modèle, idéal pour temps-réel.

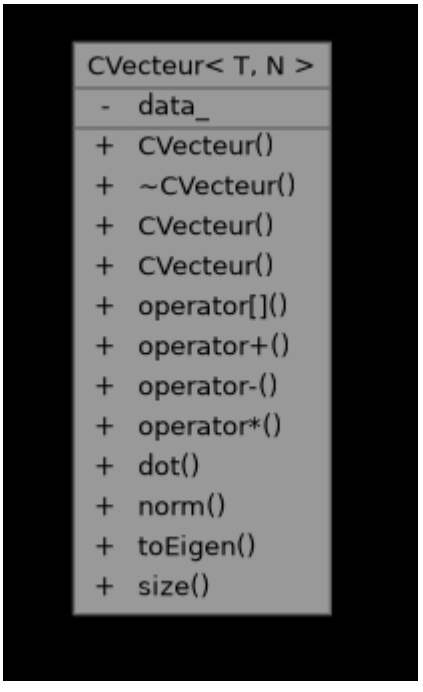
c. Le modèle de collision Pinocchio s'appuie sur hpp-fcl. Quelle différence avec le modèle cinématique, et à quoi sert computeCollisions() ?

- Cinématique : transformées SE3 des corps (positions/orientations).
- Collision : chaque corps est associé à une forme géométrique (BVH, capsule, convexe) gérée par hpp-fcl (GJK/EPA).
computeCollisions() met à jour les poses des formes via updateGeometryPlacements() puis teste chaque paire de corps déclarée dans GeometryModel : retourne distance minimale et point de contact, utilisé pour l'évitement d'obstacles et la planification de mouvement.

Diagramme UML :


CJoint
q_
qMin_
qMax_
+ CJoint()
+ ~CJoint()
+ getTransform()
+ getTypeName()
+ clone()
+ setQ()
+ getQ()
+ getQmin()
+ getQmax()
CJointPrismatic
+ CJointPrismatic()
+ getTransform()
+ getTypeName()
+ clone()
CJointRevolute
- dx_
+ CJointRevolute()
+ getTransform()
+ setdx()
+ getdx()
+ getTypeName()
+ clone()


CBras
- joints_
+ CBras()
+ CBras()
+ operator=()
+ CBras()
+ operator=()
+ ~CBras()
+ addJoint()
+ getNbJoints()
+ getJoint()
+ computeFK()
+ getQ()
+ setQ()
+ randomQ()


CVecteur< T, N >
- data_
+ CVecteur()
+ ~CVecteur()
+ CVecteur()
+ CVecteur()
+ operator[]()
+ operator+()
+ operator-()
+ operator*()
+ dot()
+ norm()
+ toEigen()
+ size()